

Routability-Driven TSV-Aware Floorplanning Methodology for Fixed-Outline 3-D ICs

Jai-Ming Lin, *Member, IEEE*, and Jung-An Yang

Abstract—Although 3-D floorplanning has been studied widely, routability which is a very important issue in modern integrated circuit (IC) designs is rarely discussed. Floorplanning in 3-D ICs is much difficult than that in 2-D ICs because of large difference in sizes between modules and through silicon vias (TSVs), which are key components in 3-D ICs. And the locations of TSVs have great impact on wirelength and routability in resulting floorplans. Hence, this paper proposes a TSV-aware 3-D floorplanning methodology which can consider wirelength and routability at the same time under the fixed-outline constraint. Unlike most of previous works which completely apply the simulated annealing algorithm, our methodology mainly apply deterministic algorithms to resolve the problem. Thus, our approach is more efficient and flexible than previous works. Experimental results have demonstrated that the proposed methodology can significantly reduce routing congestion in 3-D ICs with a slight increase in wirelength.

Index Terms—Floorplanning, physical design, three-dimensional integrated circuits, through-silicon vias.

I. INTRODUCTION

THREE-DIMENSIONAL integrated circuits (3-D ICs) have been considered as a visible solution to progress the 2-D scaling pace predicted by Moore's law. By adopting several manufacturing techniques, it is possible to realize a large amount of functionalities in a 3-D IC without increasing cost of each transistor [32].

Early 3-D ICs use through silicon vias (TSVs) to connect different tiers. TSVs can be fabricated in three different ways including the via-first approach, the via-middle approach, and the via-last approach [3]. The via-first approach makes vias before CMOS. In order to avoid potential issues in the following temperature cycles of CMOS, it has to use thermal resistance materials to implement TSV fillers. However, vias are made between CMOS and back end of line (BEOL) in the via-middle approach; hence, TSV fillers can be made by common materials such as copper. In the via-last approach, vias are made after BEOL or after the bonding process.

Manuscript received October 3, 2016; revised December 15, 2016, February 7, 2017, and March 28, 2017; accepted April 5, 2017. Date of publication April 19, 2017; date of current version October 18, 2017. This work was supported by MOST of Taiwan under Grant 105-2221-E-006-245. This paper was recommended by Associate Editor C. C.-N. Chu. (*Corresponding author: Jai-Ming Lin.*)

The authors are with the Department of Electrical Engineering, National Cheng Kung University, Tainan 701, Taiwan (e-mail: jmlin@ee.ncku.edu.tw; rogerjan918@gmail.com).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCAD.2017.2695900

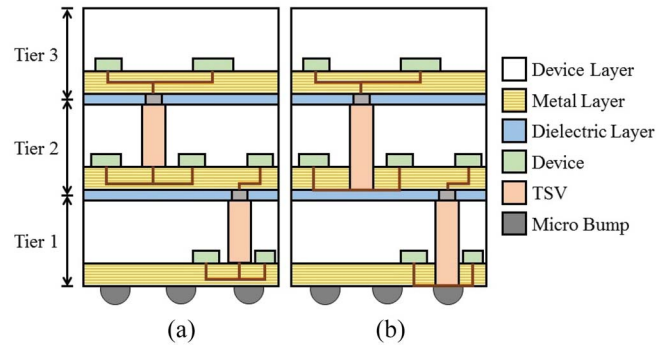


Fig. 1. Configurations of three tier 3-D ICs made by different TSV approaches. (a) Via-first (or via-middle) approach. (b) Via-last approach.

Thus, tiers can be first fabricated in a foundry and then are aligned and bonded in a packaging house. Since the diameter of a TSV is larger than that of the via-first and via-middle approaches, its alignment accuracy is smaller than the other two approaches [23]. Moreover, TSVs in the three approaches would block device layers, but only the via-last TSVs block metal layers. Fig. 1 shows an example of 3-D ICs formed by the via-first and via-middle approaches. Since the configurations of 3-D ICs formed by the via-first and via-middle approaches are similar, this paper only mentions via-first TSVs in the remaining sections.

3-D ICs have several advantages such as small wirelength, low power tier to tier communication, and high bandwidth [4]. But a high integrated density and a small chip area also bring many challenges to route nets in a 3-D IC. In order to increase communication capabilities for large-scale 3-D logic integration, massive interconnect structures are imported into 3-D chips, which lead to many wires to be routed inside a single tier or through multiple tiers. More TSVs have to be deployed for connecting those wires passing through different tiers. Since the size of a TSV is significantly large than the dimension of a standard cell, resources blocked by TSVs cannot be ignored [12]. Misplacing TSVs will result in a detour of nets or induce severe routing congestion. Hence, TSV placement becomes a key step in a 3-D chip design, and the locations of TSVs have to be planned earlier in a physical design stage. To obtain better performance, modules and TSVs must be planned simultaneously during floorplanning. In addition, a tier size is usually determined before floorplanning. To obtain a feasible solution, 3-D floorplanning also has to take account of the fixed-outline constraint.

Recently, a new manufacture technology, called monolithic integration [2], is proposed for 3-D ICs. Unlike a TSV-based 3-D IC, monolithic 3-D sequentially integrates thin device layers within a foundry and uses monolithic intertier vias (MIVs) as vertical connections for different tiers. Since sizes of MIVs are the same as regular local vias ($<100\ \mu\text{m}$), it is possible to insert a large amount of MIVs, which can effectively reduce vertical wirelength. Hence, performance and power consumption of a monolithic 3-D IC is superior than a TSV-based 3-D IC [26], [29].

A. Previous Works

Most works use the simulated annealing (SA) algorithm with a representation to handle 3-D floorplanning. These methods can be divided into two categories based on their representations: some works use the true-3-D representation and others apply the quasi-3-D representation. The true-3-D representation extends a 2-D representation to a 3-D structure by appending an additional dimension to the original 2-D representation such as 3-D slicing tree [7], 3-D CBL [27], sequence triple [33], 3-D-subTCG [37], and T-tree [38]. The true-3-D representation induces a lot of redundancy in solution space because of the additional data structure, which results in increasing runtime and space penalty [11]. The quasi-3-D representation [10], [17], [19], [20], [34] uses an array of an existing 2-D representation to construct floorplans in different device layers. These approaches first allocate modules into different layers and then apply intralayer operations and inter-layer operations to perturb solutions within a layer as well as across different layers, respectively.

In addition to using the SA-based approach, some researches use different approaches to deal with 3-D floorplanning such as 3-D-STAF [40] which applied the force-directed technique to distribute modules to 3-D space. Besides, Li *et al.* [22] used the generalized slicing tree (GST) with the enumerative packing (EP) [36] to legalize modules and TSVs in a fixed-outline after modules have been allocated to different tiers.

In recent years, more and more works [24], [25], [36] studied 2-D floorplanning with the fixed-outline constraint after Kahng [13] pointed out that the outline of an IC should be fixed rather than variable. Similar to 2-D ICs, 3-D floorplanning has to take the fixed-outline constraint into account. In order to handle the constraint in 3-D floorplanning, Chen and Yoshimura [6] extended sequence pairs (SPs) and proposed P-SP to represent floorplans in 3-D ICs. They showed the number of configurations represented by P-SP is less than SP.

Recently, several works which consider TSV assignment and the fixed-outline constraint during 3-D floorplanning [1], [17], [22], [31], [34], [35] have been proposed. However, these algorithms are mostly SA-based approaches and usually require longer runtime to get a good solution. Instead of using the SA algorithm, Li *et al.* [22] presented an efficient and effective 3-D fixed-outline floorplanner, named Co-place, to handle the problem. They first applied the dynamic algorithm to allocate modules to different tiers while minimizing total number of TSVs. Then, a GST, which is

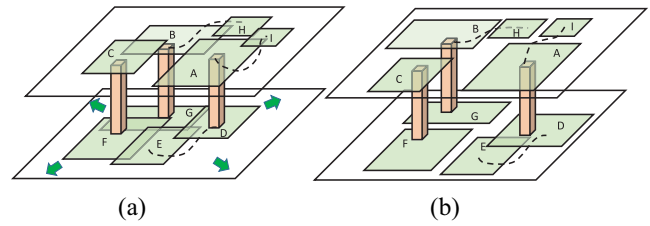


Fig. 2. Analytical-based 3-D floorplanning methodology. (a) Initial distribution of modules and TSVs. (b) Spreading of modules and TSVs to the placement region in each tier.

proposed by Yan and Chu [36], is constructed for each tier by applying the round-robin hypergraph partitioning algorithm, and feasible floorplans of all tiers are obtained by the GSTs. Finally, they arrange TSVs by the network-flow-based algorithm which further optimizes total wirelength.

Other research handles TSVs at the post-floorplanning stage. Since TSVs are not planned with modules during the floorplanning stage, it may induce a large detour when there exists no suitable whitespace for placing the associated TSV of a net. Thus, this kind of work usually requires to redistribute whitespace. To favor TSV insertion, Zhong *et al.* [39] proposed to insert whitespace to redistribute whitespace in a given floorplan, and Kim *et al.* [16] applied whitespace manipulation for those TSVs without suitable placement space.

There exist limited works discussing routability in 3-D floorplanning. Li *et al.* [21] proposed to use probabilistic analysis to estimate the routing congestion for 3-D ICs. To make the result more accuracy, the influence of TSV locations is also considered in the model. Recently, Panth *et al.* [30] discussed the routability-driven placement for standard cells in monolithic 3-D ICs. They developed a routing demand model for monolithic 3-D-ICs, and used it to develop a min-overflow partitioner which is able to enhance routability by off-loading demand from one tier to another.

B. Our Contribution

This paper proposes a routability-driven floorplanning methodology for 3-D ICs under the fixed-outline constraint. Unlike previous works using the SA algorithm which perturbs modules randomly or the iterative approach which partitions modules tier by tier, our approach optimizes locations of modules and TSVs in all tiers at the same time by the analytical-based approach [5], [25]. Since the method is much faster than SA and has a global view than the iterative approach, we can obtain better results without spending longer runtime.

Fig. 2 gives an example to reveal the good property of our methodology. Fig. 2(a) shows an initial distribution of modules and TSVs according to the analytical-based approach [5], [25], which includes nine modules that are dispersed in the two tiers of a 3-D IC. Note that module B is connected to module G in another tier by a TSV, and it also connects to module H in the same tier. Once module B is distributed to the top-left corner in the top tier, modules H and B will be pulled to the same

direction due to minimization of wirelength in the analytical formulation as shown in Fig. 2(b).

The large difference in sizes between modules and TSVs increases difficulties in distribution and legalization. An intuitive method to resolve the problem is to cluster TSVs before applying the analytical-based approach in distribution components. But the clustering approach may restrict solution quality. Hence, this paper proposes a two step approach in the global distribution (GD) stage which will distribute TSVs twice. The experimental results have demonstrated the effectiveness of the procedure. Moreover, to increase a success rate of legalization under the fixed-outline constraint, we also propose a more precise cost function during recursive partition of modules and TSVs after a GD (note that a slicing tree will be built according to the result).

Most of previous works do not consider routability in 3-D floorplanning. Due to high integration density, severe routing congestion may be induced in modern 3-D ICs. Because of higher flexibility in the proposed methodology, our approach can optimize routability and wirelength at the same time. Besides, many works use the minimum cost maximum flow algorithm to determine locations of TSVs. But runtime of the algorithm will increase significantly as the number of TSVs increases. Because initial locations of TSVs have been obtained in the first three stages in our methodology, we apply the divide-and-conquer strategy to reduce runtime in TSV assignment while maintaining a good solution.

Based on above descriptions, the characteristics of this paper are summarized in the following.

- 1) We optimize locations of modules and TSVs in all tiers at the same time based on the analytical-based approach [5], [25].
- 2) We consider wirelength and routability simultaneously during 3-D floorplanning.
- 3) To consider diversities in sizes between modules and TSVs, we propose a two step approach in the GD stage and give a better cost function for the recursive partition in the legalization stage.
- 4) We apply the divide-and-conquer approach to determine locations of TSVs which can reduce runtime without losing solution quality.

The remaining sections are organized as follows. Section II gives the problem formulation. Section III describes preliminaries for this paper. The stages of our methodology are illustrated in Sections IV–VI, respectively. Experimental results are shown in Section VII. Section VIII concludes this paper.

II. PROBLEM FORMULATION

Let $M = \{m_i | i \in \mathbb{Z}^+, 1 \leq i \leq n\}$ denote a set of n modules. The width, height, and area of a module m_i are denoted by w_i , h_i , and a_i , respectively. The aspect ratio of a module m_i is defined as w_i/h_i . Modules in M can be classified into two types: one is a hard module and the other is a soft module, where the aspect ratio of a hard module is fixed while the aspect ratio of a soft module is variable.

Given a 3-D IC with a fixed number of tiers (assume K denotes the number) and a netlist, the TSV-aware fixed-outline floorplanning is to determine the location of each module m_i or a TSV t_i , which includes its layer z_i , where $1 \leq z_i \leq K$, and its coordinate (x_i, y_i) inside a fixed outline such that no overlap between a module and a TSV, two modules, or two TSVs happens. The objective is to minimize overall wirelength and routability. This paper has three assumptions. first, the tiers of a 3-D IC are assembled by F2B bonding; second, the I/O bumps are situated below tier 1; third, only one TSV is inserted in two contiguous tiers for each 3-D net (i.e., a net that connects pins in different tiers).

III. PRELIMINARIES

Our methodology is extended from F-FM [25] which is a 2-D fixed-outline floorplanner. Hence, the section first reviews F-FM and then introduces our 3-D floorplanning design flow.

A. Methodology of F-FM

F-FM is composed of the GD stage and legalization stage. The GD stage roughly spreads modules over a specified region while the legalization stage removes overlaps of modules.

1) *Global Distribution Stage*: The GD stage applies the analytical-based approach [5] to spread modules over a placement region. A set of bins is constructed first by dividing a specified region into nonoverlapping uniform bins, where the number of bins is determined by the number of modules. Let b denote a bin. Then, the following function is solved:

$$\min W + \lambda \sum_b (D(b) - \bar{A}_b)^2 \quad (1)$$

where the first term minimizes wirelength which is denoted by W . The second term intends to uniformly distribute modules over all bins, where $D(b)$ denotes the actual area of modules in b and $\bar{A}_b$ denotes the allowable area of modules in b , which is equal to the total area of modules divided by the number of bins. Because all terms in (1) must be smooth and differentiable, the stable-log-sum-exp wirelength model [18] and the bell-shape function [28] are applied to compute W and $D(b)$, respectively. The value $D(b)$ which is computed by the bell-shape function is considered as the potential value of b . λ is a user-specified weight for the second term, whose value is initialized as follows:

$$\lambda = \frac{\sum |\partial W|}{\sum |\partial D(b)|}. \quad (2)$$

Then, (1) is solved by the conjugate gradient (CG) method with a dynamic step size [5]. The Polak–Ribiere parameter determines the step size in each iteration, and the solution of the previous iteration is taken as the initial solution for the next iteration.

2) *Legalization Stage*: To keep the geometric relation of modules in the first stage, the legalization stage first builds a slicing tree according to the distribution of modules. Then, locations and shapes of modules are determined based on the slicing tree.

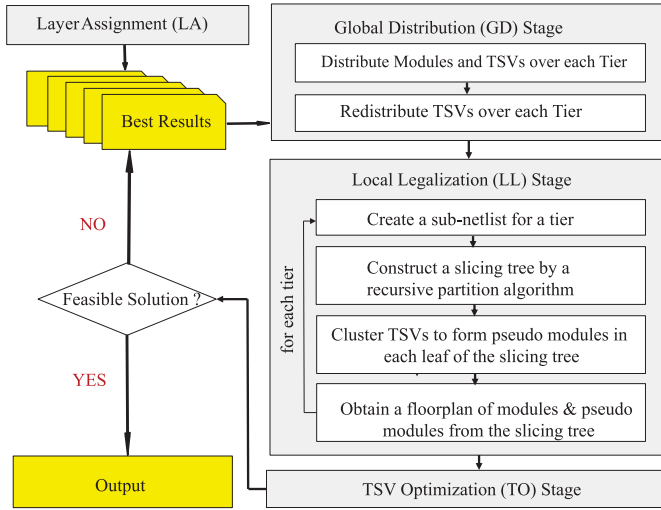


Fig. 3. Flow chart of our methodology.

A slicing tree is built according to the result of recursively bisecting modules into subregions, where a leaf in the slicing tree comprises a set of modules in a subregion while an internal node denotes the merging direction of its children. The shape curve of each leaf is generated by applying the EP [36], which is composed of a set of points, where each point represents a rectangular placement outline which can accommodate the associated modules. Then, the shape curve of an internal node is generated by bottom-up merging the two shape curves of its child nodes until the shape curve of the root is obtained. After selecting a feasible point from the shape curve of root, the shape and location of each module is determined by top-down back tracing of the slicing tree.

B. Our Methodology

Our 3-D floorplanning methodology includes four stages: 1) layer assignment (LA) stage; 2) global distribution (GD) stage; 3) local legalization (LL) stage; and 4) TSV optimization (TO) stage. Different from 2-D floorplanning, modules have to be allocated to tiers of a 3-D IC before their shapes and locations are determined. Hence, we first assign modules to tiers in the LA stage. Total module area and number of TSVs in each tier are determined after this stage. We will generate several LA results in case that infeasible solutions are obtained. Next, the GD and LL stages determine locations and shapes of modules and TSVs. Finally, exact locations of TSVs are optimized in the TO stage if a feasible solution can be obtained; otherwise, we take another LA result and repeat the procedure until a feasible solution is obtained. Fig. 3 shows the flow chart of our methodology.

IV. FIRST TWO STAGES

This section illustrates the LA and the GD stages, respectively.

A. Layer Assignment Stage

The LA stage first applies hMETIS [14] to partition modules into K parts, where K is the number of tiers in a 3-D IC.

hMETIS minimizes total number of cuts while balancing area of modules in each partition. Each cut represents a net passing through neighboring partitions, which corresponds to a TSV in a 3-D IC.

Next, the result is optimized by applying the SA algorithm. Because hMETIS only balances area of modules in each tier without considering area occupied by TSVs, it may induce an imbalanced partition. The operation of SA is to swap modules in two different tiers. The cost function is as follows:

$$\text{cost} = N_{\text{TSV}} + \sqrt{\frac{1}{K} \sum_{i=1}^K (A_i - \bar{A})^2} \quad (3)$$

where N_{TSV} denotes total number of TSVs in a 3-D IC and K is the number of tiers. A_i denotes the total area of modules and TSVs in the i th tier while $\bar{A}$ denotes the average area of modules and TSVs in a tier, which is equal to total area of modules and TSVs in all tiers dividing by K . Since the total area of TSVs in a tier may be changed after each perturbation, the values of A_i and $\bar{A}$ should be recomputed as well.

B. Global Distribution Stage

After modules have been allocated to different tiers, the GD stage simultaneously distributes modules and TSVs to placement regions in all tiers by the analytical-based approach [5], [25] while optimizing total wirelength and routability at the same time.

1) Routability-Driven Analytical Formulation:

$$\begin{aligned} \min \quad & W \\ \text{s.t.} \quad & D(b) \leq \bar{A}_b, \quad \forall b \\ & U(e) \leq C_e \quad \forall g. \end{aligned} \quad (4)$$

In addition to considering wirelength and utilization of all components in all bins as (1), this formulation also considers congestion induced by placement of components. Hence, a grid graph is built first by dividing a routing area into regular grids, where each grid is denoted by g . $U(e)$ measures the expected routing usage of a routing edge e in the grid graph and C_e is the capacity of e . The routing demand $U(e)$ is composed of the usage of nets [i.e., $\text{Usage}(e)$] and the blockage of components [i.e., $\text{blockage}(e)$] as follows:

$$U(e) = \text{Usage}(e) + \text{blockage}(e). \quad (5)$$

And C_e denotes capacity of e , and it is computed by the model proposed by Kim *et al.* [15].

After each net is decomposed into two pin nets, which is denoted by (p_1, p_2) , the usages of routing edges inside the bounding box enclosed by the two pins are updated by the following equation (without loss of generality, we assume e is a horizontal edge):

$$\begin{aligned} \text{Usage}(e) = & \frac{1}{\text{Col}(p_1, p_2)} \times f(y_{p_1}, y_e, y_{p_2}) \\ & \times \{1 - [1 - f(x_{p_1}, x_{e_1}, x_{p_2})] \times [1 - f(x_{p_1}, x_{e_2}, x_{p_2})]\} \end{aligned} \quad (6)$$

where (x_{e_1}, y_e) and (x_{e_2}, y_e) , respectively, denote the coordinate of one end of the horizontal edge e in the left side and right

side [i.e., $e = \{(x_{e1}, y_e), (x_{e2}, y_e)\}$]. Besides, $f(u, x, v)$ is the logical function defined in the following:

$$f(u, x, v) = \begin{cases} 1 & \text{if } u \leq x \leq v \text{ or } v \leq x \leq u \\ 0 & \text{else.} \end{cases} \quad (7)$$

The function returns 1 if x is in the range between u and v . The three logical functions in (6) are used to check if the horizontal edge e is covered by the bounding box. Once e is enclosed by the bounding box, the usage of e is increased by a penalty $[1/\text{Col}(p_1, p_2)]$, where $\text{Col}(p_1, p_2)$ denotes the number of columns which is covered by the bounding box in the grid graph. The expected usage of e is computed by summing up the contribution of all two-pin nets which cover e . Similarly, the expected usage $\text{Usage}(e)$ of a vertical routing edge can be computed. Because the logical function $f(u, x, v)$ is not differential, it is replaced by the sigmoid function as follows:

$$\tilde{f}(u, x, v) = \frac{1}{1 + e^{\varepsilon(u-x)(v-x)}} \quad (8)$$

where ε is a parameter controlling the precision of estimated $\tilde{f}(u, x, v)$.

To solve (4), we relax the constraints into the objective function as follows:

$$W + \alpha \sum_e (U(e) - C_e)^2 + \beta \sum_b (D(b) - \bar{A}_b)^2 \quad (9)$$

where α and β are user specified parameters which give the weights to routability and utilization in the second and third terms. (9) is solved by the CG method [8]. In the beginning, we set $\alpha = \|\nabla W\|/\|\nabla U(e)\|$ and $\beta = |\nabla W|/|\nabla D(b)|$.

The solutions of the analytical-based approach [5], [25] are affected by two factors. One is the width w_b of a bin b . The other is allowable moving distances of all components in every iteration of the CG algorithm. Components are placed around the center of each tier in the beginning. Then CG moves components outward at some distance in every iteration until the components are evenly distributed in placement regions. Moving distance of components, which is denoted by α_k , is computed by the following equation [5]:

$$\alpha_k = \frac{s \times w_b}{\|d_k\|} \quad (10)$$

where s is a user-specified scaling factor, and d_k denotes moving direction which is computed in each iteration. Based on the above equation, α_k is determined after s is set. Giving a small s would induce a better solution, but it will take longer runtime because components are moved at a shorter distance in each iteration.

2) *Flow of Global Distribution Stage*: Our GD stage is composed of two steps.

The first step simultaneously distributes modules and TSVs over placement regions according to the analytical formulation in (4). We represent a netlist by a graph $G = (V, E)$. Since modules have been assigned to tiers, each 3-D net is divided into several subnets based on the 3-D wirelength estimation model [22], where each subnet represents a 2-D net in a tier. Assume every two subnets of a 3-D net in contiguous tiers are connected by one TSV. To consider the impact of TSVs on wirelength and routability, a virtual pin is allocated to tier

$k + 1$ for each TSV in tier k , where the virtual pin does not occupy area. Hence, the location of a virtual pin has no impact on the utilization $D(b)$ of the associated bin b .

The set V is composed of modules, TSVs, and virtual pins. To ensure that a TSV and the associate virtual pin are connected vertically in a 3-D IC, we use the same variable to represent their coordinates. Through the assignment, locations of all modules and TSVs in different tiers can be optimized simultaneously by the analytical formulation. The set E consists of all 2-D subnets, and each subnet may connect to pins, TSVs, and virtual pins. Hence, W is obtained by the summation of half-perimeter wire lengths (HPWLs) of all subnets in each tier. Since this step distributes modules as well as TSVs, the bin width w_b is set as the average size of modules as follows:

$$w_b = \sqrt{\frac{\sum_{m_i \in M} a_i}{n}} \quad (11)$$

where n is the total number of modules and a_i is the area of a module m_i in M .

TSVs which are made by via-first or via-last approaches have different requirement in routing resource. Since via-first TSVs almost do not block metal routing layers, their routing demands are ignored. However, we have take account of routing demands of via-last TSVs because they will block routing layers occupied by them. Because the sizes of TSVs are too small compared to modules, we will slightly increase their areas to enhance their impacts on routability.

The second step redistributes TSVs and virtual pins, where modules are fixed at locations which are obtained in the first step. Because of large difference in sizes between modules and TSVs, TSVs may not be placed at best locations in the first step. Since a TSV is much smaller than a module, the parameters w_b and s in the analytical formulation may need to be adjusted in the second step. To explore the impact of s and w_b , we compare wirelengths and runtimes in the three Gigascale Systems Research Center (GSRC) benchmarks n100, n200, and n300 by changing their values (see Fig. 4). The x -axis in all line charts denotes w_b . The right most point in the x -axis is computed by (11). The y -axis in a upper line chart represents the resulting wirelength while it represents runtime in a lower line chart. Lines with different colors denote results obtained by setting various values of s . The figure shows that wirelength does not change dramatically as s changes. But, it usually induces better wirelength if w_b is set to a value which is smaller than the right most point. Finally, we compared the result without applying this step in the GD stage. The experimental results show that it can further reduce wirelength by 0.91% in average in the three GSRC benchmarks n100, n200, and 300 if this step is applied.

V. LEGALIZATION STAGE

The legalization stage removes overlaps of modules and TSVs without violating a good result obtained from the previous stage. Similar to F-FM, we use a slicing tree to record distribution of modules and TSVs in each tier to keep the GD stage result and get locations and shapes of modules based

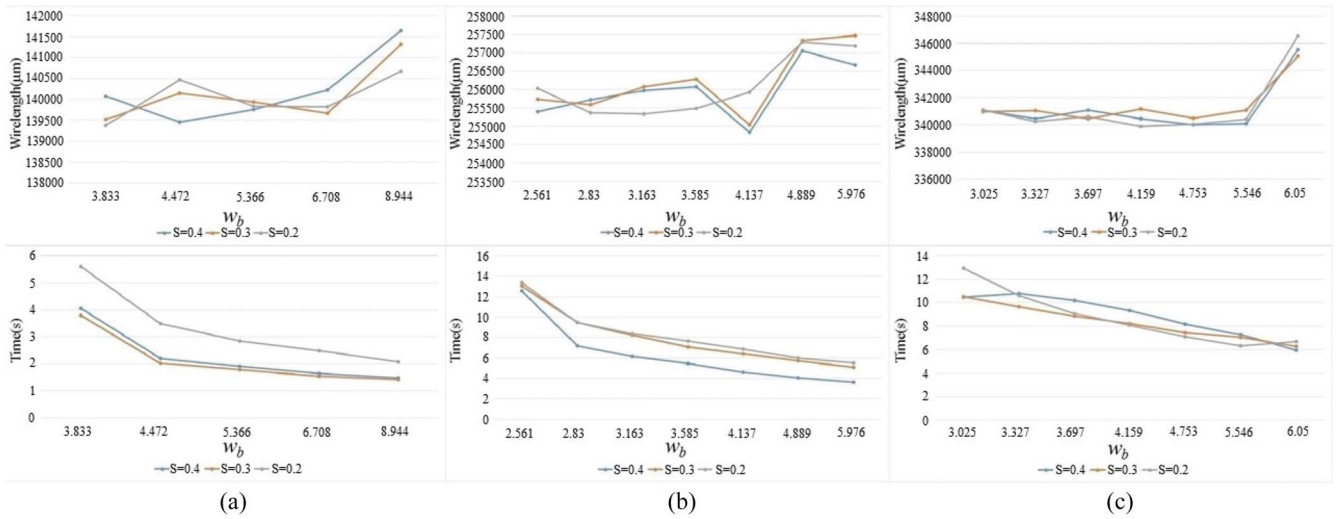


Fig. 4. Line charts for wirelength and runtime while different bin width w_b and s are given when we redistribute TSVs in the global redistribution stage for the three GSRC benchmarks. (a) n100. (b) n200. (c) n300.

on it. First, the possible shapes of floorplans for the modules in a leaf of a slicing tree are obtained by applying EP [36]. Then, the exact locations and shapes of modules in each tier are determined by the bottom-up curve merging and top-down back tracing in the slicing tree.

A. Flow of Legalization Stage

Due to existence of TSVs, difficulty of legalization in a 3-D IC increases a lot. We have to employ extra effort to process TSVs in order to increase chance to get a feasible solution. For building a slicing tree, modules and TSVs in a tier are allocated to subregions by applying a recursive partition algorithm (see Section V-B). In each iteration, modules and TSVs in a region are divided into two subregions and an internal node is added into the tree to denote the partition. Since quality of partition has a great impact on the success rate of legalization, we propose a more precise cost function in the partition to avoid misleading by small TSVs. The procedure is repeated until the number of modules and TSVs in a subregion is smaller than a user specified value.

After modules and TSVs are partitioned into subregions, we further cluster TSVs in each subregion to form *pseudo modules* (see Section V-C). Since the number of TSVs in each tier is much larger than that of modules, the depth of a slicing tree will increase if we treat each TSV as a small hard module. Hence, runtime of a program also increases. Moreover, due to large difference in sizes between modules and TSVs, it is hard to get a feasible solution under the fixed-outline constraint.

B. Recursive Partition Algorithm

Given a region P with placed modules and TSVs, a bisection is to partition them into two parts and to allocate a suitable subregion for each part.

To perform bisection, P is divided into uniform tiles along the y - (or x -) axis by a set of vertical (horizontal) cuts, where the number of tiles is determined by user. Let L denote a set of cuts. Each cut l_j in L separates modules and TSVs into two parts according to the center coordinates of the modules (or

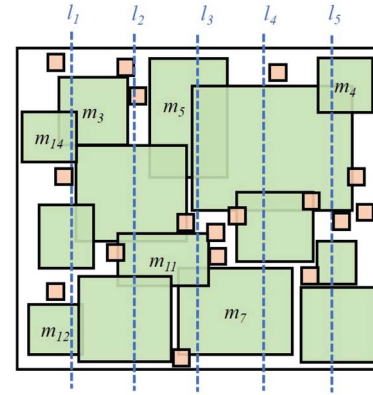


Fig. 5. Selection of a cut from five vertical cuts $\{l_1 \dots l_5\}$ for dividing the modules and TSVs in the region into two parts.

TSVs) with respect to the x - (y -) coordinate of l_j . To avoid an unbalanced bisection, any cut near the boundary of P is removed from L . The process repeats until the number of modules and TSVs in each subregion is less than a predefined number. Fig. 5 shows an example of a bisection of the modules and TSVs in a region. The region is divided into six tiles by a set $L = \{l_1, \dots, l_5\}$ of cuts, where each cut represents a possible partition of modules and TSVs.

Each cut l_j in L is given a cost and the cut with the smallest cost is selected to divide modules and TSVs into two parts. A module (or a TSV) which is passed by l_j will contribute a cost to l_j . Since a 2-D IC does not contain TSVs, Lin and Wu [25] simply used a moving distance of a module m_i as its cost. But this may induce a wrong solution when a cut is passed by many TSVs in 3-D ICs. Hence, we propose to use the moving area of m_i as the penalty as follows:

$$\hat{F}(m_i, l_j) = \begin{cases} h_i(x_i + w_i - x_j), & x_i + \frac{w_i}{2} \leq x_j \leq x_i + w_i \\ h_i(x_j - x_i), & x_i \leq x_j \leq x_i + \frac{w_i}{2} \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

where w_i and h_i denote the width and height of m_i , respectively. x_j denotes the x -coordinate of l_j . For a horizontal cut, the

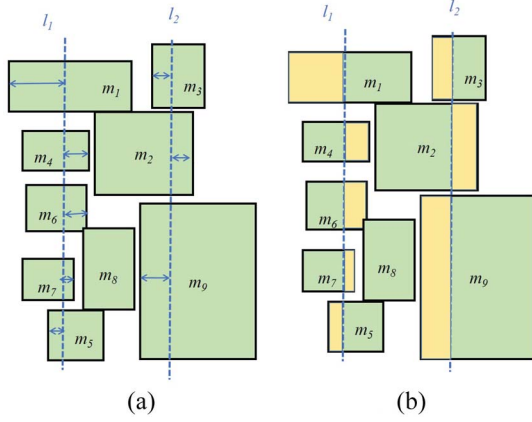


Fig. 6. Evaluation of penalty according to a total moving (a) distance of modules and (b) areas of modules.

cost function is modified by replacing w_i , h_i , and x -coordinates in (12) to h_i , w_i , and y -coordinates, respectively. For example, there exist nine modules which are passed by two vertical cuts l_1 and l_2 in Fig. 6. The vertical cut l_2 is selected if we use a moving distance as the cost [see Fig. 6(a)]. However, modules m_2 , m_3 , and m_9 have larger moving areas, which induce a larger change in a module area in each subregion. This may increase the difficulty of legalization. Instead, l_1 is a better choice since it uses a moving area as its cost as shown in Fig. 6(b).

The cost of each cut l_j in L is computed by the following function $\hat{\phi}(l_j)$:

$$\hat{\phi}(l_j) = \epsilon_j \sum_{m_i \in P} \hat{F}(m_i, l_j). \quad (13)$$

Since a cut which is close to the center of a region P can lead to a more balanced bisection than that close to the boundary of P , we introduce an additional parameter ϵ_j to scale the cost as follows (ϵ_j is defined similarly when l_j is horizontal):

$$\epsilon_j = \left(1 + \frac{\|x_c - x_j\|}{w_{\text{tile}} \cdot |L|}\right)^\gamma \quad (14)$$

where x_c is the center x -coordinate of P and $|L|$ is the number of vertical cuts in L . γ is a user specified parameter and it is a positive integer value (γ is set as 4 in our experiment). Let w_{tile} denote width of a vertical (or horizontal) tile. Assume vertical (horizontal) cuts in L are sorted by their x - (y -) coordinates in the increasing order. ϵ_j with a smaller value means that l_j is closer to the center of P , and vice versa. Fig. 7 shows the curve of ϵ_j with respect to x_j , and the center x -coordinate is x_c . With the scale factor, the cut which is close to the center of P will be selected with respect to another cut which is far from if the two cuts originally have identical costs by $\hat{F}$.

C. TSV Clustering

To maintain efficiency of the procedure EP [36], the number of modules in a subregion cannot be larger than 9. Hence, this section shows a partition-based algorithm to cluster TSVs in each subregion r such that the number of TSV clusters plus m_r is most close to 9 and is less than or equal to 9, where m_r

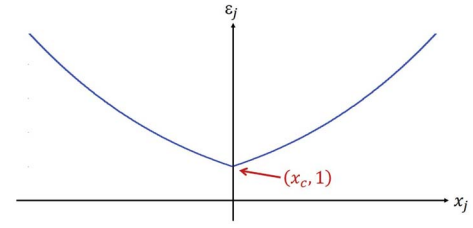


Fig. 7. Curve of ϵ_j . The x -axis denotes the x coordinate x_j of a cut l_j , where the center coordinate in the figure is x_c . The y -axis denotes the value of ϵ_j .

denotes an initial number of modules in r after the partition algorithm introduced in the last section is applied. Note that m_r must be smaller than 9.

Let T_r denote a set of TSVs in r and all TSVs in T_r form as a group in the beginning. Each time the group with the largest number of TSVs is partitioned into two groups. And the procedure continues until the terminal condition is met. TSVs in each group form a pseudo module, and the modules and pseudo modules in r constitute a leaf in a slicing tree.

For each module in the subregion, three vertical (horizontal) cuts are generated from its center, left (bottom) boundary, and right (top) boundary. Let L_r denote a set of vertical (horizontal) cuts formed by all modules, where each vertical (horizontal) cut l_i ($l_i \in L_r$) partitions all TSVs in r into two groups: the left and right (bottom and top) groups, which are denoted by T_L and T_R (T_B and T_U), respectively. For each TSV, it is arranged to T_L (T_B) if its center coordinate lies in the left (bottom) side of l_i ; otherwise, it is arranged to T_R (T_U).

Let b_r represent a bounding box which encloses all TSVs in T_r . If any cut is out of or too close to the boundary of b_r , it is excluded from T_r to avoid an unbalanced partition. The following function is used to evaluate the quality of a cut l_i and the cut with the smallest value is selected:

$$\phi(l_i) = \sigma(T_L) + \sigma(T_R) \quad (15)$$

where $\sigma(T_L)$ [$\sigma(T_R)$] represents the standard deviations for the distances between l_i and all TSVs in T_L (or T_R). Without loss of generality, we assume that the direction of l_i in (15) is vertical. By, respectively, replacing T_L and T_R with T_B and T_U , $\phi(l_i)$ associated to a horizontal cut l_i can be calculated. $\sigma(T_L)$ is computed by the following equation ($\sigma(T_R)$ can be calculated similarly):

$$\sigma(T_L) = \sqrt{\frac{\sum_{t_q \in T_L} (d_{q,l_i} - \mu_x)^2}{|T_L|}} \quad (16)$$

where t_q denotes a TSV in T_L , d_{q,l_i} denotes the distance between t_q and l_i , and μ_x denotes the average distance from l_i to all TSVs in T_L . $|T_L|$ is the total number of TSVs in T_L . A small value of $\sigma(T_L)$ implies that TSVs in T_L are close to each other along x -axis. If both $\sigma(T_L)$ and $\sigma(T_R)$ are small, it means that TSVs in either T_L or T_R are placed at near locations along x -axis.

Fig. 8 shows an example for TSV clustering. As shown in Fig. 8(a), eleven vertical cuts (i.e., $L_r = \{l_1, \dots, l_{11}\}$) are generated according to the modules in the region. To prevent an unbalanced partition, we first neglect l_1 and l_{11} . Then, one

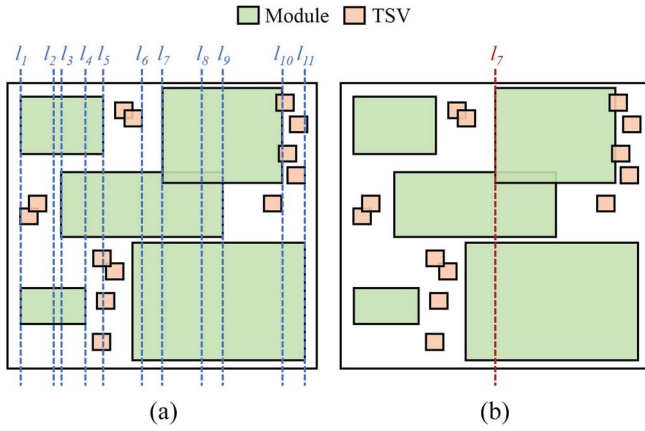


Fig. 8. TSV clustering approach. (a) Set $L_r = \{l_1, \dots, l_{11}\}$ of vertical cuts is formed by the modules. (b) l_7 is selected to separate TSVs in the region into two groups.

cut in L_r is selected according to (15). Assume l_7 is selected, which separates TSVs into two groups as shown in Fig. 8(b). Since number of modules plus TSV groups is 7 which is still less than 9, the two groups will be further partitioned, and the left group which has a larger size is partitioned first.

VI. OPTIMIZATION OF TSVs

This section proposes a divide-and-conquer approach to determine locations of TSVs according to the minimum cost maximum flow algorithm which can optimize wirelength and routability at the same time.

A. Construction of Network-Flow Graph

Since the locations of TSVs in each tier can be determined separately, we determine their locations from bottom tier to top tier. First, a network-flow graph $G_k = (V_k, E_k)$ for the k th tier is constructed, $k = 1$ to K . All TSVs in a tier form a set V_t of TSV nodes, where v_i^t ($v_i^t \in V_t$) denotes a TSV. Besides, the placement region of a tier is divided into several slots, and each slot can fit in one TSV. All slots form a set V_s of slot nodes, where v_j^s ($v_j^s \in V_s$) denotes a slot. Because the restriction of TSVs inside the associated pseudo modules may lead to worse wirelength and routability, we allow TSVs to be placed inside the associated pseudo modules as well as whitespace in a tier. Finally, a source node v_{src} and a sink node v_{trg} are added to G_k .

The information of $G_k = (V_k, E_k)$ is as follows.

- 1) The node set $V_k = \{v_{src}\} \cup \{v_{trg}\} \cup V_t \cup V_s$.
- 2) The edge set $E_k = \{(v_{src}, v_i^t) \mid v_i^t \in V_t\} \cup \{(v_j^s, v_{trg}) \mid v_j^s \in V_s\} \cup \{(v_i^t, v_j^s) \mid v_i^t \in V_t, v_j^s \in V_s\}$.
- 3) The capacity Φ of an edge: $\Phi(v_{src}, v_i^t) = 1$, $\Phi(v_j^s, v_{trg}) = 1$, and $\Phi(v_i^t, v_j^s) = 1$.
- 4) The cost Ψ of an edge: $\Psi(v_{src}, v_i^t) = 0$, $\Psi(v_j^s, v_{trg}) = 0$, and $\Psi(v_i^t, v_j^s) = \Psi_{i,j}$.

The cost of an edge (v_i^t, v_j^s) denotes the penalty for placing a TSV t_i into a slot s_j , which is computed by the following function:

$$\Psi^{i,j} = \Psi_{HPWL}^{i,j} + \delta \times \Psi_{routability}^{i,j} \quad (17)$$

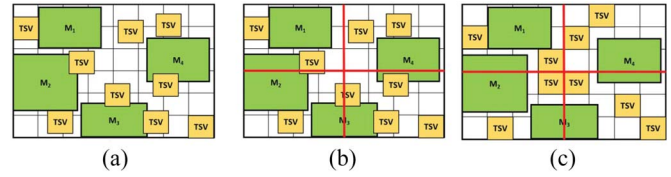


Fig. 9. (a) Initial distribution of modules and TSVs. (b) Division of a placement region into four subregions. (c) Optimization of locations of TSVs in each subregion, respectively.

where

$$\begin{aligned} \Psi_{HPWL}^{i,j} &= f_w(\text{SB}[b_i^k, s_j]) + f_w(\text{SB}[b_i^{k+1}, s_j]) \\ \Psi_{routability}^{i,j} &= f_c(t_i, s_j). \end{aligned} \quad (18)$$

There exist two terms in (17). The first term is about wirelength and the second term is about routability. b_i^k (b_i^{k+1}) in the equation denotes the bounding box of the subnet connected to t_i in tier k ($k+1$). Note that k is in the range between 1 to $K-1$. Since two contiguous subnets are connected by one TSV, b_i^k (b_i^{k+1}) is formed by the modules associated to the subnet except the first tier (i.e., $k=1$) which may be formed by modules and I/O bumps. Let $\text{SB}[b_i^k, s_j]$ denote the smallest bounding box which encloses one point in b_i^k and s_j if s_j is not inside b_i^k ; otherwise, the dimension of the bounding box is 0×0 . $f_w(\text{SB}[b_i^k, s_j])$ returns the half perimeter of $\text{SB}[b_i^k, s_j]$. $f_w(\text{SB}[b_i^{k+1}, s_j])$ can be defined, similarly.

The second term in (17) computes the congestion penalty induced by placement of TSVs. Since via-first TSVs do not block routing metal layers, we only need to consider this term when via-last TSVs are used. Placement of a via-last TSV t_i in a tier k will induce additional congestion in the associated tier. But it has no impact in the tier $k+1$. Therefore, a congestion map for each tier is built according to the model [21], that a routing region is divided into regular grids. Since modules have been placed in the stage, we can build a Steiner tree for each subnet by applying FLUTE [9] and the congestion value of each grid is determined according to the result. $f_c(t_i, s_j)$ is determined according to the congestion value of a grid which covers s_j when we try to place t_i at s_j .

B. Iterative Algorithm to Reduce Runtime

The minimum cost maximum flow algorithm becomes inefficient when the numbers of slots and TSVs become larger. Since a better distribution of TSVs has been obtained in the previous stage, we do not need to apply the algorithm for the whole placement region. Instead, each tier is divided into regular subregions with proper sizes, and TSVs inside a subregion are legalized separately. For example, Fig. 9(a) shows an initial distribution of modules and TSVs. If the region is divided into 2×2 subregions in Fig. 9(b), we only need to place TSVs in each subregion into the corresponding feasible slots. Fig. 9(c) shows the resulting placement of TSVs.

Fig. 10 demonstrates efficiency of our divide-and-conquer approach by applying it to IBM circuits. The x -axis denotes the number of subregions that a placement region is divided.

TABLE I
COMPARISON OF OUR METHODOLOGY WITH CO-PLACE FOR SOFT MODULE BENCHMARKS

Cir.	Co-place [22]				Ours							
	# of TSVs	WL (μm)	Suc. rate %	Times (s)	Layer Assignment and TSV Count as [22]			Our Layer Assignment				
					WL (μm)	Suc. rate %	Times (s)	# of TSVs	WL (μm)	Suc. rate %	Times (s)	
n100	505	142500	-	3.19	128998	100	3.79	455	130145	100	37.38 + 5.33 = 42.71	
n200	1043	254757	-	3.42	231184	100	8.12	969	241208	100	20.17 + 7.93 = 28.1	
n300	1244	363384	-	4.65	317137	100	13.67	1024	322329	100	37.65 + 12.11 = 49.76	
n100	677	129057	-	1.54	117423	100	3.67	660	119398	100	30.68 + 7.21 = 37.89	
n200	1572	240392	-	3.85	213154	100	9.32	1442	219386	100	15.81 + 10.88 = 26.69	
n300	1758	342535	-	4.86	290996	100	15.48	1443	302198	100	30.64 + 23.75 = 54.39	
Nor.	1	1	-	1	0.88	1	2.51	0.88	0.9	1	9.77	

TABLE II
COMPARISON OF OUR METHODOLOGY WITH CO-PLACE FOR HARD MODULE BENCHMARKS

Cir.	Co-place [22]				Ours							
	# of TSVs	WL (μm)	Suc. rate %	Times (s)	Layer Assignment and TSV Count as [22]			Our Layer Assignment				
					WL (μm)	Suc. rate %	Times (s)	# of TSVs	WL (μm)	Suc. rate %	Times (s)	
n100	505	146070	100	1.61	139442	100	2.25	455	140082	99	37.38 + 4.05 = 41.43	
n200	1043	266419	100	2.25	246219	100	6.74	969	255399	100	20.17 + 6.61 = 26.78	
n300	1244	380638	100	2.99	336756	100	12.71	1024	341122	99	37.65 + 10.44 = 48.09	
n100	677	133980	95	2.64	129059	99	2.34	660	131793	94	30.68 + 5.94 = 36.62	
n200	1572	250829	97	4.97	228875	100	7.65	1442	234484	97	15.81 + 9.42 = 25.23	
n300	1758	350980	100	3.38	313044	100	14.8	1443	323975	99	30.64 + 21.61 = 52.25	
Nor.	1	1	1	1	0.91	1.01	2.6	0.88	0.93	0.99	12.91	

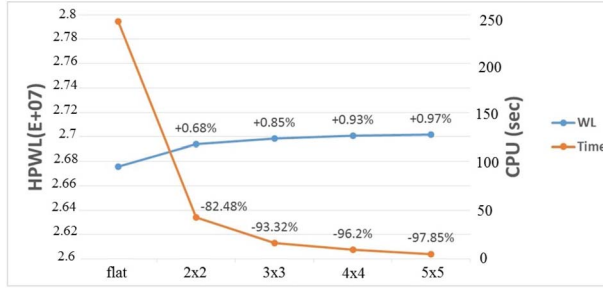


Fig. 10. Wirelength versus runtimes in the divide-and-conquer approach.

The y-axis in the left side denotes wirelength and that in the right side denotes runtime, which are the average values of the results from all circuits. The figure reveals that it takes a lot of time to apply the minimum cost maximum flow algorithm if circuits are flat, but this induces the shortest wirelength. When a placement region is divided into 2×2 , runtime reduces significantly while there exists only slight increase in wirelength.

VII. EXPERIMENTAL RESULTS

Our 3-D floorplanner was implemented in C++ programming language and run on an Intel Xeon Quad Core 2.27-GHz Linux workstation with 90-GB memory.

A. Comparison With Co-Place

This section compares our 3-D floorplanning methodology with Co-place [22] based on GSRC benchmarks. Similar to Co-place, 100 testcases are generated for each benchmark by assigning I/O bumps randomly. Hence, resulting wirelength and runtime are average values over 100 runs. The aspect ratio of each tier is set to 1 with 15% whitespace. The size of a

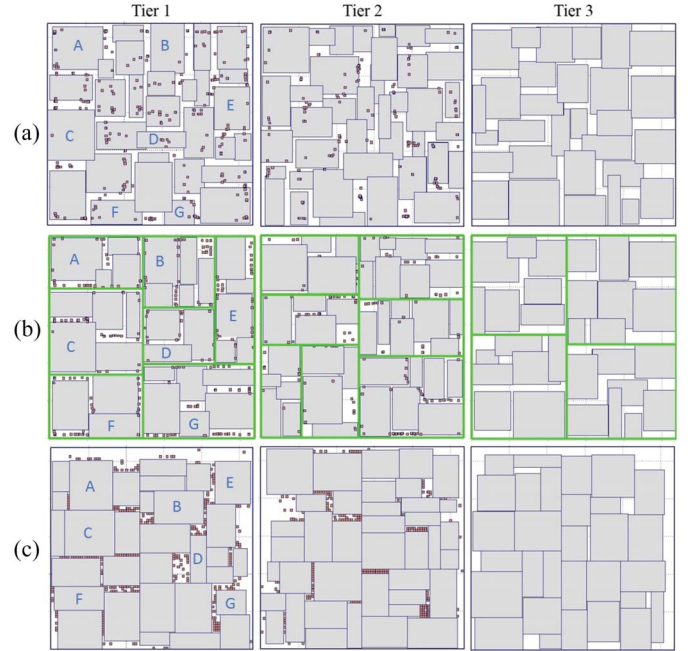


Fig. 11. Resulting floorplans generated by our methodology in different stages for n100 in a 3-D IC with three tiers. Result after the (a) GD stage and (b) recursive partition. (c) Final result.

TSV is $3 \times 3 \mu\text{m}$ in each testcase, where an elementary unit is set to $1 \mu\text{m}$.

Tables I and II show the results for soft modules and hard modules, respectively. In both tables, the results of Co-place are listed in columns 2–5 while our results are shown in columns 6–12, where our results can be further divided into two parts: 1) columns 6–8 show the results when we take the LA results from Co-place as our input and 2) the results shown in columns 9–12 are completely based on our methodology

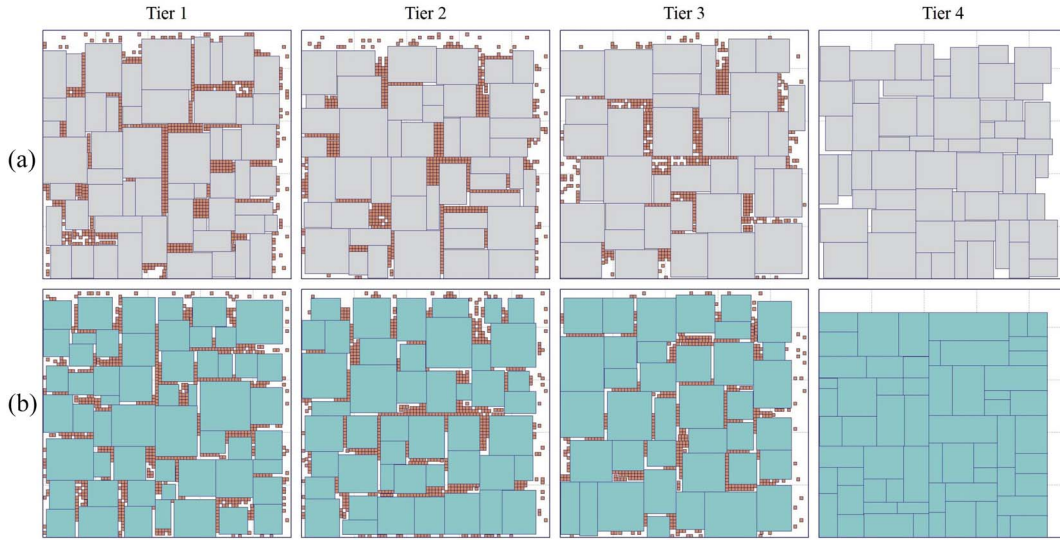


Fig. 12. Resulting floorplans of n200 in a 3-D IC with four tiers. The gray rectangles represent hard modules while the small orange squares represent TSVs. (a) Hard modules. (b) Soft modules.

TABLE III

OUR RESULTS ON WHEN THE CHIP ASPECT RATIOS BECOME 2 AND 3, RESPECTIVELY. ROWS 2–4 AND 5–7 SHOW THE RESULTS FOR SOFT MODULES AND HARD MODULES, RESPECTIVELY

Cir.	# of TSVs	Aspect Ratio 2			Aspect Ratio 3		
		WL (μm)	Suc. rate %	Times (s)	WL (μm)	Suc. rate %	Times (s)
n100	455	140250	100	9.3	151339	100	14.4
n200	969	256837	100	25.5	276367	100	41.9
n300	1024	338241	100	32.5	364090	100	53.1
n100	455	154625	94	7.5	166796	96	12.5
n200	969	274925	100	23.6	297414	96	39.9
n300	1024	362675	98	30.2	389012	100	50.6

including the LA method. The first column of the tables shows the names of the circuits. Columns 4, 7, and 11 show the success rates in 100 testcases. Since Co-place does not show the success rates for soft modules, column 4 in Table I is marked by “-.” Our methodology achieves 100% success rates for soft modules. The success rate for hard modules is slightly lower than that for soft modules, but it still can achieve 99% in average. Our wirelength is much shorter than Co-place no matter in the cases of hard modules or in the cases of soft modules. Moreover, we use less number of TSVs than co-place. Column 12 shows the runtime, which is composed of runtimes spent in the LA stage and other stages, respectively. Our runtime is longer than co-place because the analytical-based approach [5], [25] is adopted in our methodology. But the worst runtime is less than one minute, which is still very fast. Table III shows the floorplanning results of 3-D ICs with three tiers when the aspect ratios of chip outlines become 2 and 3, respectively. Rows 2–4 show the results for soft modules while rows 5–7 show the results for hard modules. In general, wirelength becomes worse when the chip aspect ratio becomes larger since it is more difficult to fit all modules inside such chip outlines. But we still can obtain 100% success rate for soft modules and the runtime is less than one minute for all cases.

Fig. 11, respectively, shows the floorplans in different stages by our methodology. The initial distribution of modules and

TABLE IV
INFORMATION OF IBM BENCHMARKS

Cir.	Modules			# of Nets	# of Pads
	Hard	Soft	Total		
ibm01	246	665	911	5829	246
ibm07	291	829	1120	15047	287
ibm13	424	530	954	27011	490
ibm18	285	658	943	42200	272

TSVs after the GD stage is shown in Fig. 11(a). Fig. 11(b) shows the results after the recursive partition algorithm is applied (note that a slicing tree is built according to the result in each tier). The final result is shown in Fig. 11(c). The figure shows that final locations of modules are similar to their initial distribution after the GD stage. For the sake of clarity, we label some modules in the first tier in Fig. 11(a)–(c). It reveals that our methodology not only can obtain a good initial solution in the GD state, but the result also can be kept after the LL and TO stages. Fig. 12 shows the resulting floorplans of n200 in a 3-D IC with four tiers. All modules in Fig. 12(a) are hard while all modules in Fig. 12(b) are soft.

B. Results Considering Routability and Wirelength

This section demonstrates capability of our 3-D floorplanning methodology in routability. The experiment uses IBM benchmarks as testcases because their sizes are much larger than GSRC. Moreover, they provide necessary information for estimating routing congestion. We only choose four benchmarks because IBM benchmarks are originally designed in 2-D ICs which may contain some modules whose dimensions are extremely larger than other modules. It will lead to an imbalance partition if such benchmarks are designed in 3-D ICs. The information of the benchmarks is shown in Table IV. There exist nine routing layers in total, where each module blocks five metal layers. The edge capacity in each metal layer may be different.

TABLE V
RESULTS BASED ON OUR METHODOLOGY WHICH OPTIMIZES WIRELENGTH

Cir.	W/O routability							
	WL ($10^3 \mu\text{m}$)	Time (s)	congestion in first layer			congestion in second layer		
			AVG.	MAX	MAX _{1%}	AVG.	MAX	MAX _{1%}
ibm01	2750.8	51.3	0.97	41.5	18.5	1.95	71.2	52.8
ibm07	14131.1	173.6	1.21	26.0	11.7	1.87	67.2	50.6
ibm13	33128.2	202.6	0.96	4.0	2.1	2.14	97.4	61.7
ibm18	68169.3	549.8	4.26	71.0	35.4	2.48	50.1	23.7
Nor.	1	1	1	1	1	1	1	1

TABLE VI
RESULTS OF OUR ROUTABILITY-DRIVEN METHODOLOGY APPLIED TO A 3-D IC USING THE VIA-FIRST APPROACH AND THE VIA-LAST APPROACH, RESPECTIVELY

Cir.	W/ routability															
	VIA-first approach									VIA-last approach						
	WL ($10^3\mu\text{m}$)	Time (s)	congestion in first layer			congestion in second layer			WL ($10^3\mu\text{m}$)	Time (s)	congestion in first layer			congestion in second layer		
			AVG.	MAX	MAX _{1%}	AVG.	MAX	MAX _{1%}			AVG.	MAX	MAX _{1%}	AVG.	MAX	MAX _{1%}
ibm01	2801.8	59	0.93	18.1	13.2	1.78	70.5	52.4	2805.1	53	0.94	19.1	14.1	1.78	61.3	50.7
ibm07	14408.7	179	1.25	27.3	13.4	1.72	61.4	38.9	14459.0	175	1.21	23.0	11.2	1.85	65.0	41.0
ibm13	35476.4	225	0.76	4.0	1.9	2.07	32.0	17.9	34220.2	202	0.74	2.0	1.0	2.13	99.3	62.1
ibm18	71063.8	560	3.38	58.8	36.5	1.43	43.0	31.5	70451.1	558	3.84	46.2	33.2	2.48	23.1	10.9
Nor.	1.04	1.04	0.85	0.76	0.96	0.83	0.72	0.74	1.03	1.01	0.91	0.63	0.88	0.97	0.87	0.87

TABLE VII
COMPARISON OF THE WIRELENGTH OPTIMIZATION METHODOLOGY WITH THE ROUTABILITY-DRIVEN METHODOLOGY IN A 3-D IC USING THE VIAS-LAST APPROACH WHEN ASPECT RATIO BECOMES 2

Cir.	W/O routability									W/ routability								
	WL ($10^3 \mu\text{m}$)	Time (s)	congestion in first layer			congestion in second layer			WL ($10^3 \mu\text{m}$)	Time (s)	congestion in first layer			congestion in second layer			WL ($10^3 \mu\text{m}$)	Time (s)
			AVG.	MAX	MAX _{1%}	AVG.	MAX	MAX _{1%}			AVG.	MAX	MAX _{1%}	AVG.	MAX	MAX _{1%}		
ibm01	2767.3	84	1.10	29.0	20.2	1.75	79.1	56.6	2823.7	83	0.91	16.1	11.9	1.71	78.8	58.1	2823.7	83
ibm07	15226.2	268	1.19	10.6	89.7	2.12	150	90.1	15603.2	287	0.89	26.6	12.8	2.09	47.2	31.5	15603.2	287
ibm13	32806.6	346	5.99	156	44.7	2.48	236	145	34676.9	361	1.48	17.0	7.3	2.81	115	60.8	34676.9	361
ibm18	74046.1	844	2.31	70.6	43.6	4.34	173	107	76196.8	860	2.27	63.0	22.8	1.80	136	65.2	76196.8	860
Nor.	1	1	1	1	1	1	1	1	1.03	1.03	0.52	0.46	0.27	0.78	0.59	0.54	1.03	1.03

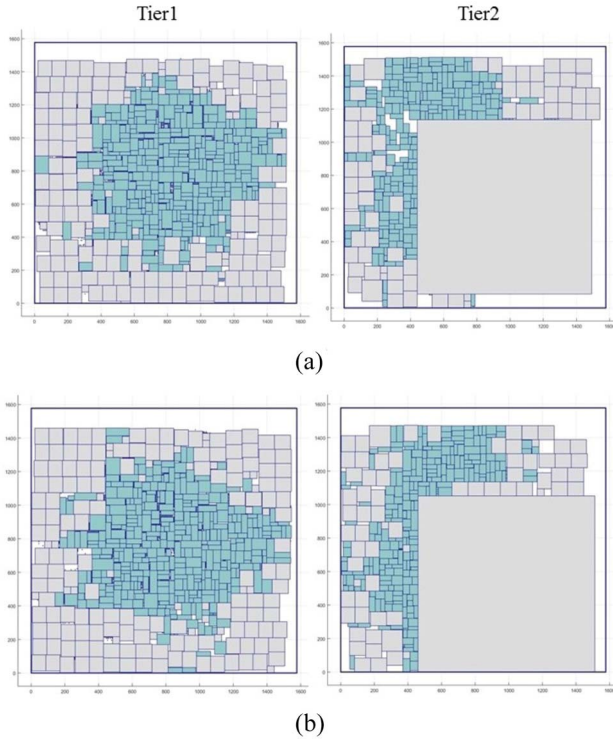


Fig. 13. Resulting floorplans of ibm01 in a 3-D IC with two tiers based on (a) wirelength optimization methodology and (b) routability-driven methodology.

The experimental results are shown in Tables V and VI, respectively. For comparison, Table V shows the results in which only wirelength is optimized while the results shown

in Table VI are optimized in wirelength and routability simultaneously. The results in Table VI can be further divided into two parts: columns 2–8 show the results based on the via-first approach while columns 9–15 show the results according to the via-last approach. Since the via-first approach does not block routing layers, it gives better routability compared to that using the via-last approach. The routing congestion shown in the table is measured by the model [21]. To view routing congestions of resulting floorplans in different ways, we use various functions to estimate the values which include an average congested value, the most congested value, and the first 1% average value, which are denoted by AVG., MAX, and MAX_{1%} in the tables, respectively. Compared to the results in Table V, wirelength in the via-first approach (via-last approach) is increased by 4% (3%) when the routability-driven methodology is applied shown in Table VI. But the AVG. congestions are decreased by 15% (9%) and 17% (3%) in the first layer and in the second layer, respectively. Furthermore, the MAX. congestions are significantly decreased by 24% (37%) and 28% (13%) in the first layer and in the second layer, respectively. The results demonstrate that better routability can be obtained in the via-first approach compared to the via-last approach.

In Table VII, we also compare the wirelength optimized methodology (shown in columns 2–8) with the routability-driven methodology based on via-last TSVs (shown in columns 9–15) when the aspect ratio of a 3-D IC is set to 2. Although wirelength slightly increases 3% in the routability-driven methodology, congestion is greatly relieved especially in the values of MAX_{1%}. Fig. 13 shows the

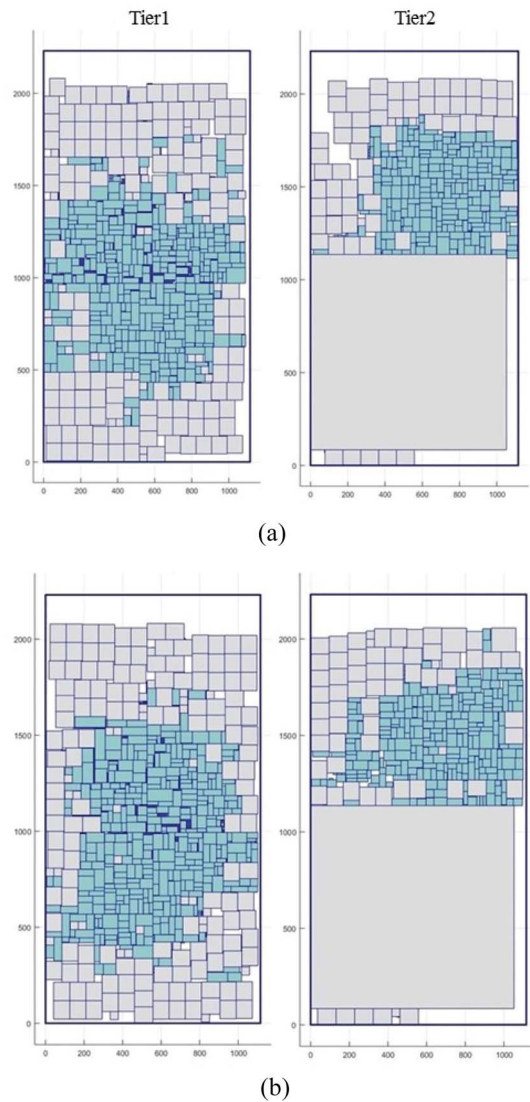


Fig. 14. Resulting floorplans of ibm01 in a 3-D IC with two tiers where the aspect ratio is 2 based on (a) wirelength optimization methodology and (b) routability-driven methodology.

resulting floorplans of ibm01 in a 3-D IC with aspect ratio 1. Fig. 13(a) shows the result according to the wirelength optimization methodology while Fig. 13(b) shows the result by the routability-driven methodology. Similarly, Fig. 14 shows the resulting floorplans of ibm01 when the aspect ratio of a 3-D IC is set to 2.

VIII. CONCLUSION

In this paper, we have proposed a 3-D floorplanning methodology under the fixed-outline constraint, in which wirelength and routability are considered at the same time. The experimental results have demonstrated the capability of the proposed methodology. In the future, we will extend the methodology to consider thermal effect which is one of the most important issues in 3-D ICs.

REFERENCES

- [1] M. A. Ahmed, S. Mohapatra, and M. Chrzanowska-Jeske, "3D floorplanning with nets-to-TSVs assignment," in *Proc. ICECS*, Marseilles, France, Dec. 2014, pp. 578–581.

- [2] P. Batude *et al.*, "Advances in 3D CMOS sequential integration," in *Proc. IEEE IEDM*, Baltimore, MD, USA, Dec. 2009, pp. 1–4.
- [3] E. Beyne, "The 3-D interconnect technology landscape," *IEEE Design Test*, vol. 33, no. 3, pp. 8–20, Jun. 2016.
- [4] J. Burns *et al.*, "Three-dimensional integrated circuits for low-power, high-bandwidth systems on a chip," in *Proc. IEEE ISSCC*, 2001, pp. 268–269.
- [5] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1228–1240, Jul. 2008.
- [6] S. Chen and T. Yoshimura, "Multi-layer floorplanning for stacked ICs: Configuration number and fixed-outline constraints," *Integr. VLSI J.*, vol. 43, no. 4, pp. 378–388, Sep. 2010.
- [7] L. Cheng, L. Deng, and M. D. F. Wong, "Floorplanning for 3-D VLSI design," in *Proc. ASP-DAC*, Shanghai, China, Jan. 2005, pp. 405–411.
- [8] E. K. P. Chong and S. H. Zak, *An Introduction to Optimization*. New York, NY, USA: Wiley, Nov. 2013.
- [9] C. Chu and Y.-C. Wong, "FLUTE: Fast lookup table based rectilinear Steiner minimal tree algorithm for VLSI design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 1, pp. 70–83, Jan. 2008.
- [10] J. Cong, J. Wei, and Y. Zhang, "A thermal-driven floorplanning algorithm for 3D ICs," in *Proc. ICCAD*, San Jose, CA, USA, Nov. 2014, pp. 306–313.
- [11] R. Fischbach, J. Lienig, and J. Knechtel, "Investigating modern layout representations for improved 3D design automation," in *Proc. GVLIS*, Lausanne, Switzerland, May 2011, pp. 337–342.
- [12] V. Gerousis, "Physical design implementation for 3D IC: Methodology and tools," in *Proc. ISPD*, San Francisco, CA, USA, Mar. 2010, p. 57.
- [13] A. B. Kahng, "Classical floorplanning harmful?" in *Proc. ISPD*, San Diego, CA, USA, 2000, pp. 207–213.
- [14] G. Karypis and V. Kumar, *hMETIS, A Hypergraph Partitioning Package Version 1.5.3*. Accessed on Apr. 5, 2014. [Online]. Available: <http://glaros.dtc.umn.edu/gkhome/metis/hmetis/download>
- [15] D.-H. Kim, K. Athikulwongse, and S.-K. Lim, "A study of through-silicon-via impact on the 3D stacked IC layout," in *Proc. ICCAD*, San Jose, CA, USA, 2009, pp. 674–680.
- [16] D.-H. Kim, R. O. Topaloglu, and S.-K. Lim, "Block-level 3D IC design with through-silicon-via planning," in *Proc. ASP-DAC*, Sydney, NSW, Australia, Jan. 2012, pp. 335–340.
- [17] J. Knechtel, E. F. Y. Young, and J. Lienig, "Planning massive interconnects in 3-D chips," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 11, pp. 1808–1821, Nov. 2015.
- [18] M. Kuwano and Y. Takashima, "Stable-LSE based analytical placement with overlap removable length," in *Proc. SASIMI*, 2010, pp. 115–120.
- [19] J. H. Y. Law and E. F. Y. Young, "Multi-bend bus driven floorplanning," *Integr. VLSI J.*, vol. 41, no. 2, pp. 306–316, Feb. 2008.
- [20] Z. Li *et al.*, "Hierarchical 3-D floorplanning algorithm for wirelength optimization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 53, no. 12, pp. 2637–2646, Dec. 2006.
- [21] W. Li, J. Kim, and J.-W. Chong, "A novel congestion estimation model and congestion aware floorplan for 3D ICs," in *Proc. ICIMTR*, Malacca, Malaysia, May 2012, pp. 199–204.
- [22] C.-R. Li, W.-K. Mak, and T.-C. Wang, "Fast fixed-outline 3-D IC floorplanning with TSV co-placement," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 21, no. 3, pp. 523–532, Mar. 2013.
- [23] C.-T. Lin, D.-M. Kwai, Y.-F. Chou, T.-S. Chen, and W.-C. Wu, "CAD reference flow for 3D via-last integrated circuits," in *Proc. ASP-DAC*, Taipei, Taiwan, Jan. 2010, pp. 187–192.
- [24] J.-M. Lin and Z.-X. Hung, "UFO: Unified convex optimization algorithms for fixed-outline floorplanning considering pre-placed modules," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 7, pp. 1034–1044, Jul. 2011.
- [25] J.-M. Lin and J.-H. Wu, "F-FM: Fixed-outline floorplanning methodology for mixed-size modules considering voltage-island constraint," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 33, no. 11, pp. 1681–1692, Nov. 2014.
- [26] Y.-J. Lee, D. Limbrick, and S.-K. Lim, "Power benefit study for ultra-high density transistor-level monolithic 3D ICs," in *Proc. DAC*, Austin, TX, USA, 2013, pp. 1–10.
- [27] Y. Ma, X. Hong, S. Dong, and C. K. Cheng, "3D CBL: An efficient algorithm for general 3D packing problems," in *Proc. MWSCAS*, Covington, KY, USA, Aug. 2005, pp. 1079–1082.
- [28] W. C. Naylor, R. Donnelly, and L. Sha, "Non-linear optimization system and method for wire length and delay optimization for an automatic electric circuit placer," U.S. Patent 6301 693, 2001.

- [29] S. Panth, K. Samadi, Y. Du, and S.-K. Lim, "High-density integration of functional modules using monolithic 3D-IC technology," in *Proc. ASP-DAC*, Yokohama, Japan, Jan. 2013, pp. 681–686.
- [30] S. Panth, K. Samadi, Y. Du, and S.-K. Lim, "Placement-driven partitioning for congestion mitigation in monolithic 3D IC designs," in *Proc. ISPD*, Petaluma, CA, USA, Jan. 2014, pp. 47–54.
- [31] A. Quiring, M. Olbrich, and E. Barke, "Improving 3D-floorplanning using smart selection operations in meta-heuristic optimization," in *Proc. 3DIC*, San Francisco, CA, USA, Oct. 2013, pp. 1–6.
- [32] D. Stow, I. Akgun, R. Barnes, P. Gu, and Y. Xie, "Cost analysis and cost-driven IP reuse methodology for SoC design based on 2.5D/3D integration," in *Proc. ICCAD*, Nov. 2016, pp. 1–6.
- [33] H. Tamazaki, K. Sakanushi, S. Nakatake, and Y. Kajitani, "The 3D-packing by meta data structure and packing heuristics," *IEICE Trans. Fundam.*, vol. E83-A, no. 4, pp. 639–645, Apr. 2000.
- [34] M.-C. Tsai, T.-C. Wang, and T.-T. Hwang, "Through-silicon via planning in 3-D floorplanning," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 8, pp. 1448–1457, Aug. 2011.
- [35] L. Xiao, S. Sinha, J. Xu, and E. F. Y. Young, "Fixed-outline thermal-aware 3D floorplanning," in *Proc. ASP-DAC*, Taipei, Taiwan, Jan. 2010, pp. 561–567.
- [36] J. Z. Yan and C. Chu, "DeFer: Deferred decision making enabled fixed-outline floorplanning algorithm," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 29, no. 3, pp. 367–381, Mar. 2010.
- [37] P.-H. Yuh, C.-L. Yang, Y.-W. Chang, and H.-L. Chen, "Temporal floorplanning using 3D-subTCG," in *Proc. ASP-DAC*, Yokohama, Japan, Jan. 2004, pp. 725–730.
- [38] P.-H. Yuh, C.-L. Yang, and Y.-W. Chang, "Temporal floorplanning using the T-tree formulation," in *Proc. ICCAD*, San Jose, CA, USA, Nov. 2004, pp. 300–305.
- [39] W. Zhong, S. Chen, and T. Yoshimura, "Whitespace insertion for through-silicon via planning on 3-D SoCs," in *Proc. ISCAS*, Paris, France, May 2010, pp. 913–916.
- [40] P. Zhou *et al.*, "3D-STAF: Scalable temperature and leakage aware floorplanning for three-dimensional integrated circuits," in *Proc. ICCAD*, San Jose, CA, USA, Nov. 2007, pp. 590–597.

Jai-Ming Lin (M'08) received the B.S., M.S., and Ph.D. degrees in computer science from National Chiao Tung University, Hsinchu, Taiwan, in 1996, 1998, and 2002, respectively.

He is an Associate Professor with the Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan. From 2002 to 2007, he was an Assistant Project Leader with the CAD Team, Realtek Corporation, Hsinchu. His current research interests include physical design for 3-D-ICs, design automation for analog ICs, and low power designs.

Jung-An Yang received the B.S. degree in electrical engineering from National Chung Hsing University, Taichung, Taiwan, in 2015. He is currently pursuing the Post-Graduate degree in electrical engineering with Nation Cheng Kung University, Tainan, Taiwan.

His current research interest includes physical design for 3-D-ICs floorplanning.